

Chapter 5

Association Analysis (Part 1)

Overview

- Association rule definition
- The Apriori Algorithm
- Compact representations of mining results
- Reading assignment: pages 357-405

Market Basket Analysis

- Principal application of association rules is market basket analysis
- It models supermarket data and discovers associations between different products (or items) that are bought together by customers.
- E.g.,
 - {bread} $\Rightarrow$ {milk}
 - {diaper} $\Rightarrow$ {beer}
 - {onion dip, salsa} $\Rightarrow$ {tortilla chip}
antecedent *consequence*

Data Model

- We consider a (market basket) dataset as a collection of *transactions* (or transactional records).
- Each transaction is represented by a set of *items* (e.g., {milk, bread, detergent, rice}), also commonly called an *itemset*.
- Note: for *binary* association rules mining, the quantity of each item bought is not considered.

Notation

- D = dataset
- I = set of all items
- N = total number of transactions in D
- T = a transaction in D
- $a, b, c, A, B, C, \dots$ = items
- $X, Y, Z, \dots$ = sets of items (*itemsets*)
- note: every transaction is an itemset

Definition

- Given a non-empty itemset X , we say that a transaction T in D *contains* X iff $X \subseteq T$
 - “ T *contains* X ” or “ T *supports* X ”
- The *support count* of X , denoted by $\text{sup}(X)$, is the number of transactions in D that contain X .
- If an itemset X contains k items, we call X a *k -itemset*.

A toy example
dataset D

Tid	a	b	c	d	e	f
1	1	1	0	0	0	1
2	0	0	1	1	1	0
3	1	1	1	0	0	0
4	1	1	0	0	1	1
5	0	1	0	1	1	0
6	0	0	1	1	0	0
7	1	0	0	1	0	0
8	1	1	0	0	0	0
9	1	1	0	0	0	0
10	0	0	1	1	1	1
11	0	1	1	1	1	0
12	0	1	0	1	0	0
13	1	0	0	0	0	0
14	0	1	0	0	0	0
15	0	1	0	1	1	0
16	0	1	1	1	0	0
17	0	1	0	1	0	0
18	1	1	0	0	1	0
19	0	1	1	1	1	0
20	1	1	1	0	1	0
21	0	1	1	1	1	0
22	0	1	1	1	0	0
23	0	0	0	0	1	0
24	1	0	0	0	0	0
25	1	1	1	1	0	0

Example

- $N = 25$
- $I = \{a, b, c, d, e, f\}$
- The first transaction is the itemset $\{a, b, f\}$
- transaction 20 is $\{a, b, c, e\}$, which contains the itemset $\{a, b, e\}$
- $\text{sup}(\{a, b, e\}) = 3$ (transactions 4, 18, 20)
- $\text{sup}(\{c, d\}) = 9$
- $\{c, d\}$ is a 2-itemset

Association Rules

- An association rule is an induction rule of the form:
 - $X \Rightarrow Y$  *If X occurs, then Y is likely to occur*
- Requirements:
 - X and Y are *non-empty*, *disjoint* itemsets. (I.e., $X \neq \emptyset$; $Y \neq \emptyset$; $X \cap Y = \emptyset$)
 - *support* condition:
 - $\frac{\text{sup}(X \cup Y)}{N} \geq \rho_s$
 - *confidence* condition:
 - $\frac{\text{sup}(X \cup Y)}{\text{sup}(X)} \geq \rho_c$

Usefulness and Certainty

- ρ_s = support threshold
- The *support* condition ensures that the mined rules concern items that occur in a good portion of the transactions. Hence, the rules are of *sufficient interest* or importance to the analysts.
- Requirement:

$$\frac{\text{sup}(X \cup Y)}{N} \geq \rho_s$$



fraction of transactions that mention all the items in the rule.

Usefulness and Certainty

- ρ_c = confidence threshold
- The ratio $\text{sup}(X \cup Y) / \text{sup}(X)$ gives the conditional probability, $P(Y|X)$, that itemset Y occurs in a transaction given that itemset X occurs in the same transaction. The *confidence* condition ensures that the mined rules are of *high confidence*.
- Requirement:

$$P(Y|X) = \frac{\text{sup}(X \cup Y)}{\text{sup}(X)} \geq \rho_c$$

Example

In practice, the support threshold is much smaller than this.

- suppose $\rho_s = 20\%$, and $\rho_c = 65\%$
- w.r.t. D , consider the rule:
 - $\{f\} \Rightarrow \{a, b\}$
 - $\text{sup}(\{a, b, f\}) = 2$
 - $\text{sup}(\{f\}) = 3$
 - $N = 25$
 - $\text{sup}(\{a, b, f\}) / N = 2/25 = 8\% < \rho_s$
 - $\text{sup}(\{a, b, f\}) / \text{sup}(\{f\}) = 2/3 = 67\% > \rho_c$
 - The rule satisfies the confidence condition but not the support condition.

How to find rules?

- Step 1: find all the *frequent* itemsets
 - A non-empty itemset S is frequent iff $\text{sup}(S) \geq N \times \rho_s$.
 - Note that any valid rule must have all the items mentioned in the rule form a frequent itemset.
- Step 2: *generate rules* from frequent itemsets
 - For each frequent itemset S found in Step (1):
 - partition S into two non-empty disjoint subsets, X and Y (i.e., $X \neq \emptyset$; $Y \neq \emptyset$; $X \cap Y = \emptyset$; $X \cup Y = S$)
 - check if the rule $X \Rightarrow Y$ satisfies the confidence condition (i.e., if $\text{sup}(X \cup Y) / \text{sup}(X) \geq \rho_c$)
if so, report the rule $X \Rightarrow Y$
 - repeat this step for all possible partitioning of S

Example

- Let $\rho_s = 20\%$. Step 1 involves finding all frequent itemsets X s.t.
 - $\text{sup}(X) \geq \rho_s \times N = 20\% \times 25 = 5$.
- From D , the sets of all frequent itemsets are:

itemset	support	itemset	support
{a}	11	{b, d}	10
{b}	18	{c, d}	9
{c}	11	{c, e}	6
{d}	14	{d, e}	7
{e}	11	{c, d, e}	5
{a, b}	8	{b, c}	8
{b, e}	8	{b, d, e}	5
{b, c, d}	6		

Example

- A frequent itemset that contains only one item does not generate any rule.
- The 3-itemset $\{c, d, e\}$, for example, derives 6 *candidate rules*:

rule	confidence	rule	confidence
$\{c\} \Rightarrow \{d, e\}$	$5/11 = 45\%$	$\{d, e\} \Rightarrow \{c\}$	$5/7 = 71\%$
$\{d\} \Rightarrow \{c, e\}$	$5/14 = 36\%$	$\{c, e\} \Rightarrow \{d\}$	$5/6 = 83\%$
$\{e\} \Rightarrow \{c, d\}$	$5/11 = 45\%$	$\{c, d\} \Rightarrow \{e\}$	$5/9 = 56\%$

Example

- If the confidence threshold (ρ_c) is set to 65%, then only 2 rules are valid:

rule	confidence	rule	confidence
$\{c\} \Rightarrow \{d, e\}$	$5/11 = 45\%$	$\{d, e\} \Rightarrow \{c\}$	$5/7 = 71\%$
$\{d\} \Rightarrow \{c, e\}$	$5/14 = 36\%$	$\{c, e\} \Rightarrow \{d\}$	$5/6 = 83\%$
$\{e\} \Rightarrow \{c, d\}$	$5/11 = 45\%$	$\{c, d\} \Rightarrow \{e\}$	$5/9 = 56\%$

Example

- After considering all the frequent itemsets, the set of valid rules (that satisfy the confidence condition) are:

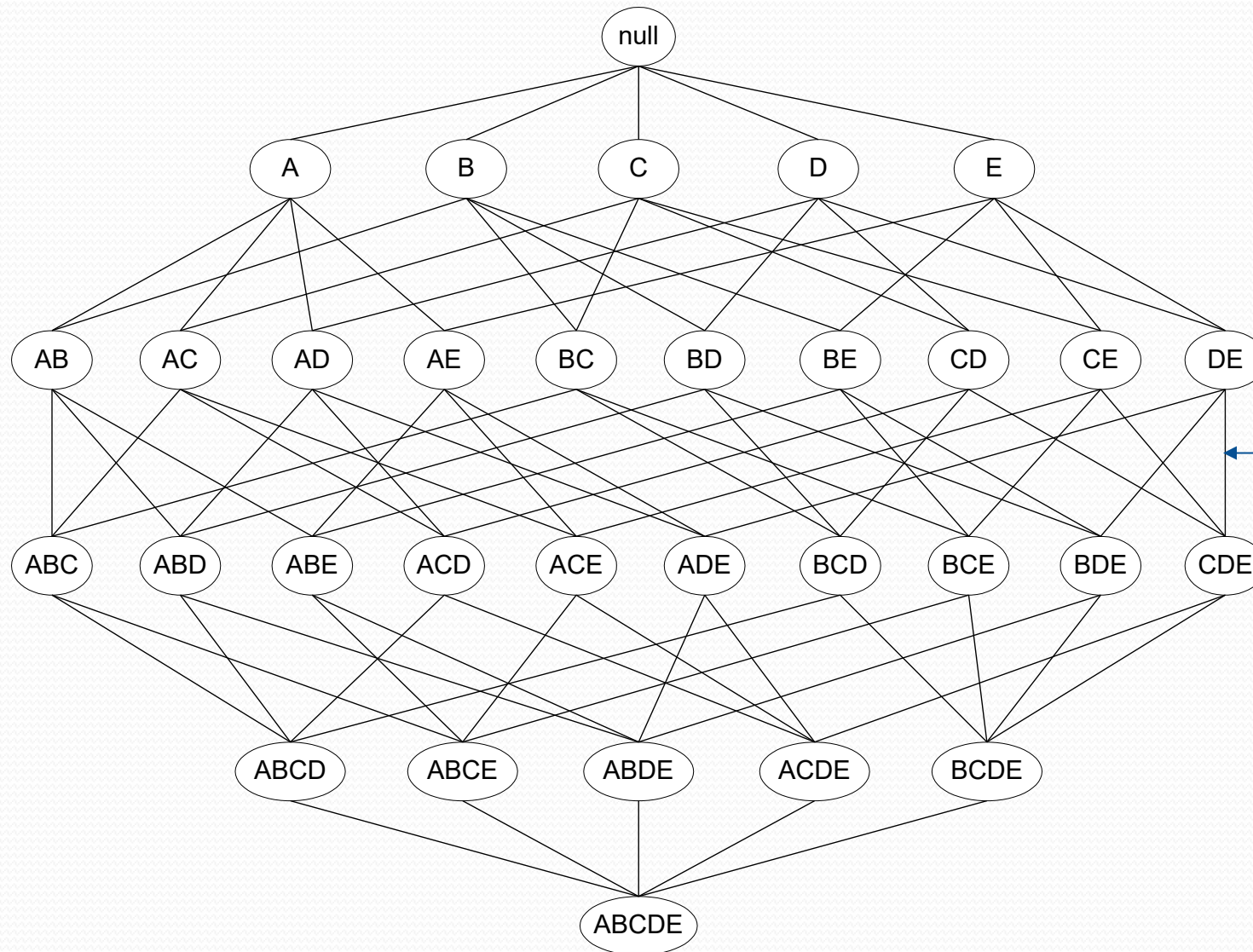
rule	confidence	rule	confidence
$\{a\} \Rightarrow \{b\}$	73%	$\{d, e\} \Rightarrow \{c\}$	71%
$\{c\} \Rightarrow \{b\}$	73%	$\{c, e\} \Rightarrow \{d\}$	83%
$\{e\} \Rightarrow \{b\}$	73%	$\{d, e\} \Rightarrow \{b\}$	71%
$\{d\} \Rightarrow \{b\}$	71%	$\{b, c\} \Rightarrow \{d\}$	75%
$\{c\} \Rightarrow \{d\}$	82%	$\{c, d\} \Rightarrow \{b\}$	67%

Generating Rules from Frequent Itemsets

- Step 2 (i.e., generating rules with sufficient confidence from frequent itemsets) is relatively straightforward.
- A frequent k -itemset generates $2^k - 2$ possible rules.
- Checking the confidence of these rules is easy, provided that the supports of all the subsets of the frequent itemsets are available.
- Since k is usually small for typical applications, an exhaustive approach for generating (and checking) rules from frequent itemsets is usually not a problem.

Finding Frequent Itemsets and Counting Their Supports

- Step 1 could be expensive.
- If there are m different items in a dataset, then there are $2^m - 1$ possible non-empty itemsets.
- E.g., if a supermarket carries 10,000 items, then there are $2^{10000} - 1$ non-empty itemsets.
- Checking which itemsets are frequent and finding their supports using a brute-force approach is impractical.



Given m items, there are $2^m - 1$ possible non-empty candidate itemsets. We can arrange them as a *lattice* structure.

← subset relation

The Apriori Algorithm

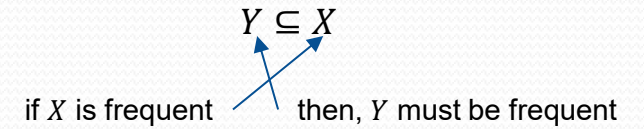
- Proposed independently by
 - Agrawal, Srikant
 - Mannila, Toivonen and Verkamo
- The Apriori property:
 - If X is a frequent itemset, then any non-empty subset of X is also frequent.
- Example: with our dataset D , $\{c, d, e\}$ is a frequent itemset ($\text{sup}(\{c, d, e\}) = 5$).

We note that all non-empty subsets of $\{c, d, e\}$ are frequent in D .

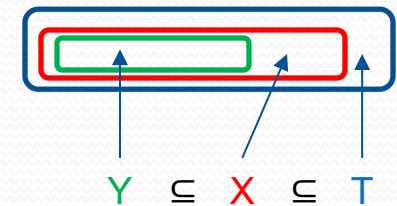
The Apriori Algorithm

itemset	support	itemset	support
$\{c\}$	11	$\{c, e\}$	6
$\{d\}$	14	$\{d, e\}$	7
$\{e\}$	11	$\{c, d, e\}$	5
$\{c, d\}$	9		

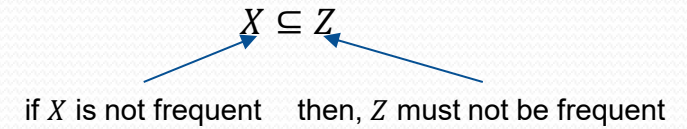
The Apriori Property



- If X is a frequent itemset, then any non-empty subset of X is also frequent.
- Proof:
 - $\text{sup}(X)$ = number of transactions in D that contain X
 - If a transaction T in D contains X , we have $X \subseteq T$.
For any $Y \subseteq X$, we have $Y \subseteq T$.
So, T also contains Y
 - Therefore, $\text{sup}(Y) \geq \text{sup}(X)$
 - Since $\text{sup}(X) \geq \rho_s \times N$, $\text{sup}(Y)$ must be $\geq \rho_s \times N$ as well
 - Hence, Y is frequent

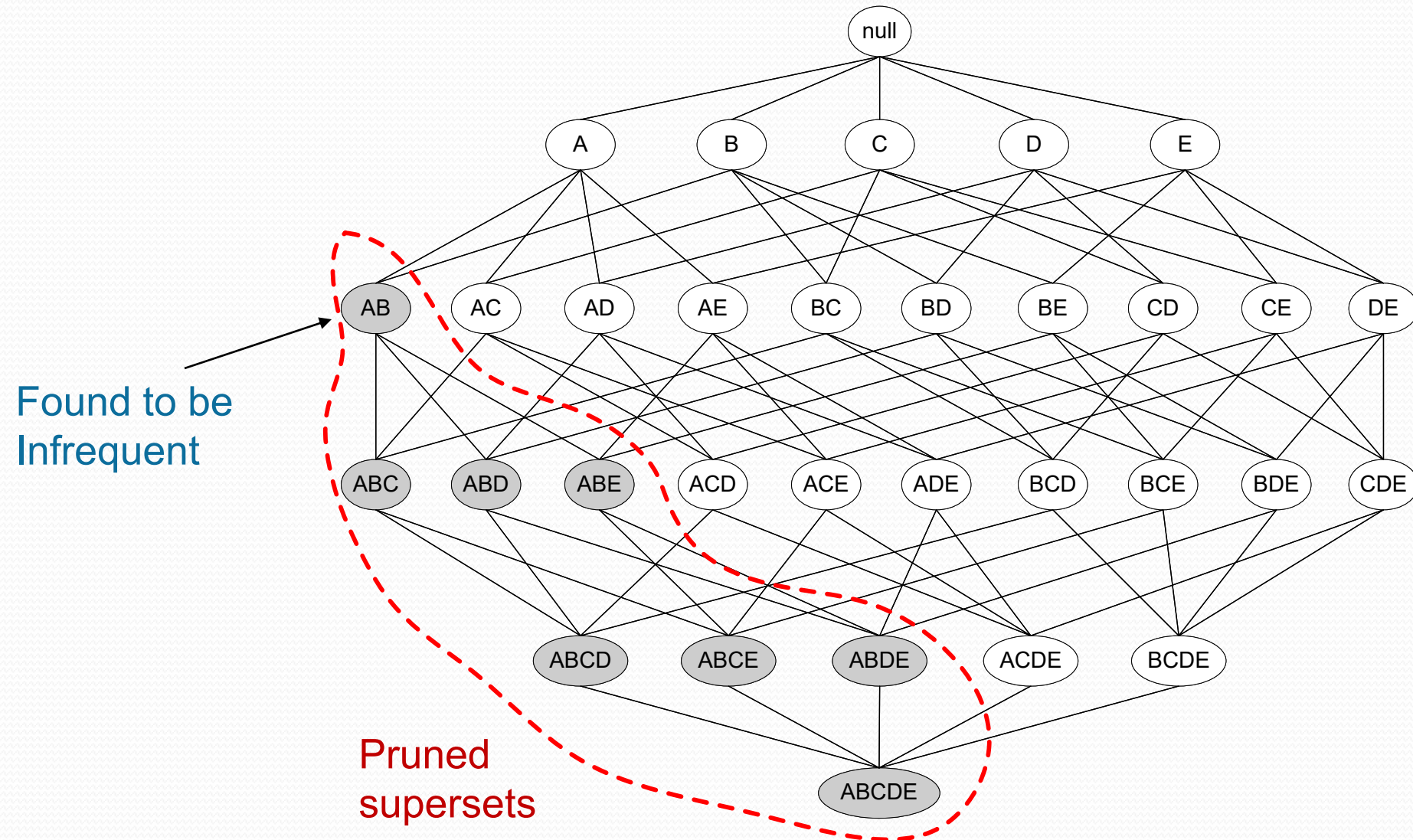


The Apriori Property



- Conversely, if an itemset X is not frequent, then any superset of X must not be frequent either.
- Therefore, if we know that an itemset X is not frequent, we do not need to check any superset of X and find its support count.
- The Apriori Algorithm makes use of the Apriori property to prune the large number of itemsets to a manageable number.
- Examples: $2^m - 1$
 - if we know that {charcoal} is not frequent, then we don't need to check the support of {charcoal, milk}, etc.

Illustrating Apriori Principle



The Apriori Algorithm

- iterative
- notation:
 C_i = set of all candidate i -itemsets
 L_i = set of all frequent i -itemsets
- 1st iteration:
 - C_1 = set of all 1-itemsets (e.g., $\{\{a\}, \{b\}, \{c\}, \{d\}, \{e\}\}$)
 - scan the dataset once to find all frequent 1-itemsets (i.e., L_1)
- 2nd iteration:
 - construct a candidate set C_2 . An itemset X is in C_2 if it contains 2 items and that all size-1 subsets of X are in L_1 .
 - scan the dataset once to find which itemsets in C_2 are frequent. Those frequent 2-itemsets are put into L_2 .

The Apriori Algorithm

- i^{th} iteration:
 - construct a *candidate set* C_i . An itemset X is in C_i if it contains i items and that all size- $(i-1)$ subsets of X are in L_{i-1} .
 - scan the dataset once to find out which itemsets in C_i are frequent. Those frequent i -itemsets are put into L_i
- Apriori terminates when a C_i generated is empty
- The set of frequent itemsets is the union of all the L_i 's found in the iterations.

Tid	a	b	c	d	e	f
1	1	1	0	0	0	1
2	0	0	1	1	1	0
3	1	1	1	0	0	0
4	1	1	0	0	1	1
5	0	1	0	1	1	0
6	0	0	1	1	0	0
7	1	0	0	1	0	0
8	1	1	0	0	0	0
9	1	1	0	0	0	0
10	0	0	1	1	1	1
11	0	1	1	1	1	0
12	0	1	0	1	0	0
13	1	0	0	0	0	0
14	0	1	0	0	0	0
15	0	1	0	1	1	0
16	0	1	1	1	0	0
17	0	1	0	1	0	0
18	1	1	0	0	1	0
19	0	1	1	1	1	0
20	1	1	1	0	1	0
21	0	1	1	1	1	0
22	0	1	1	1	0	0
23	0	0	0	0	1	0
24	1	0	0	0	0	0
25	1	1	1	1	0	0

C_1

{a}	{b}	{c}	{d}	{e}	{f}
0	0	0	0	0	0

Tid	a	b	c	d	e	f
1	1	1	0	0	0	1
2	0	0	1	1	1	0
3	1	1	1	0	0	0
4	1	1	0	0	1	1
5	0	1	0	1	1	0
6	0	0	1	1	0	0
7	1	0	0	1	0	0
8	1	1	0	0	0	0
9	1	1	0	0	0	0
10	0	0	1	1	1	1
11	0	1	1	1	1	0
12	0	1	0	1	0	0
13	1	0	0	0	0	0
14	0	1	0	0	0	0
15	0	1	0	1	1	0
16	0	1	1	1	0	0
17	0	1	0	1	0	0
18	1	1	0	0	1	0
19	0	1	1	1	1	0
20	1	1	1	0	1	0
21	0	1	1	1	1	0
22	0	1	1	1	0	0
23	0	0	0	0	1	0
24	1	0	0	0	0	0
25	1	1	1	1	0	0

{a}	{b}	{c}	{d}	{e}	{f}
1	1	0	0	0	1

Tid	a	b	c	d	e	f
1	1	1	0	0	0	1
2	0	0	1	1	1	0
3	1	1	1	0	0	0
4	1	1	0	0	1	1
5	0	1	0	1	1	0
6	0	0	1	1	0	0
7	1	0	0	1	0	0
8	1	1	0	0	0	0
9	1	1	0	0	0	0
10	0	0	1	1	1	1
11	0	1	1	1	1	0
12	0	1	0	1	0	0
13	1	0	0	0	0	0
14	0	1	0	0	0	0
15	0	1	0	1	1	0
16	0	1	1	1	0	0
17	0	1	0	1	0	0
18	1	1	0	0	1	0
19	0	1	1	1	1	0
20	1	1	1	0	1	0
21	0	1	1	1	1	0
22	0	1	1	1	0	0
23	0	0	0	0	1	0
24	1	0	0	0	0	0
25	1	1	1	1	0	0



{a}	{b}	{c}	{d}	{e}	{f}
1	1	1	1	1	1

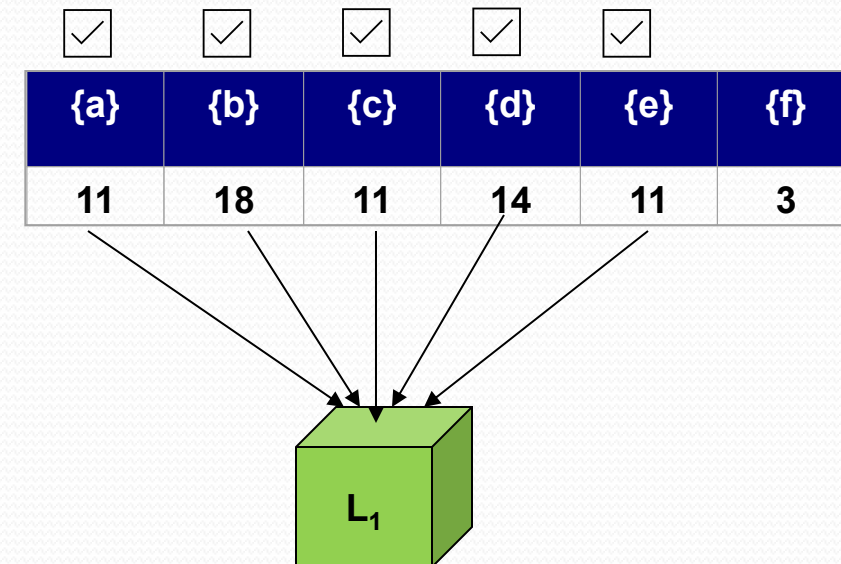
Tid	a	b	c	d	e	f
1	1	1	0	0	0	1
2	0	0	1	1	1	0
3	1	1	1	0	0	0
4	1	1	0	0	1	1
5	0	1	0	1	1	0
6	0	0	1	1	0	0
7	1	0	0	1	0	0
8	1	1	0	0	0	0
9	1	1	0	0	0	0
10	0	0	1	1	1	1
11	0	1	1	1	1	0
12	0	1	0	1	0	0
13	1	0	0	0	0	0
14	0	1	0	0	0	0
15	0	1	0	1	1	0
16	0	1	1	1	0	0
17	0	1	0	1	0	0
18	1	1	0	0	1	0
19	0	1	1	1	1	0
20	1	1	1	0	1	0
21	0	1	1	1	1	0
22	0	1	1	1	0	0
23	0	0	0	0	1	0
24	1	0	0	0	0	0
25	1	1	1	1	0	0

{a}	{b}	{c}	{d}	{e}	{f}
11	18	11	14	11	3



Tid	a	b	c	d	e	f
1	1	1	0	0	0	1
2	0	0	1	1	1	0
3	1	1	1	0	0	0
4	1	1	0	0	1	1
5	0	1	0	1	1	0
6	0	0	1	1	0	0
7	1	0	0	1	0	0
8	1	1	0	0	0	0
9	1	1	0	0	0	0
10	0	0	1	1	1	1
11	0	1	1	1	1	0
12	0	1	0	1	0	0
13	1	0	0	0	0	0
14	0	1	0	0	0	0
15	0	1	0	1	1	0
16	0	1	1	1	0	0
17	0	1	0	1	0	0
18	1	1	0	0	1	0
19	0	1	1	1	1	0
20	1	1	1	0	1	0
21	0	1	1	1	1	0
22	0	1	1	1	0	0
23	0	0	0	0	1	0
24	1	0	0	0	0	0
25	1	1	1	1	0	0

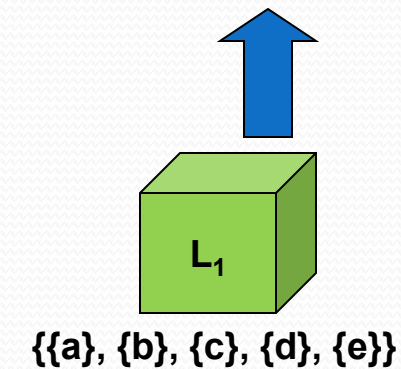
Assume a support threshold $\rho_s = 20\%$
threshold count = 5



Tid	a	b	c	d	e	f
1	1	1	0	0	0	1
2	0	0	1	1	1	0
3	1	1	1	0	0	0
4	1	1	0	0	1	1
5	0	1	0	1	1	0
6	0	0	1	1	0	0
7	1	0	0	1	0	0
8	1	1	0	0	0	0
9	1	1	0	0	0	0
10	0	0	1	1	1	1
11	0	1	1	1	1	0
12	0	1	0	1	0	0
13	1	0	0	0	0	0
14	0	1	0	0	0	0
15	0	1	0	1	1	0
16	0	1	1	1	0	0
17	0	1	0	1	0	0
18	1	1	0	0	1	0
19	0	1	1	1	1	0
20	1	1	1	0	1	0
21	0	1	1	1	1	0
22	0	1	1	1	0	0
23	0	0	0	0	1	0
24	1	0	0	0	0	0
25	1	1	1	1	0	0

C_2

{a, b}	{a, c}	{a, d}	{a, e}	{b, c}
0	0	0	0	0
{b, d}	{b, e}	{c, d}	{c, e}	{d, e}
0	0	0	0	0

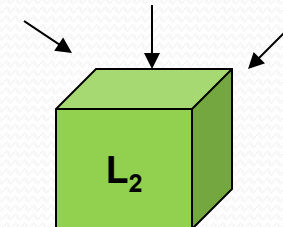


Tid	a	b	c	d	e	f
1	1	1	0	0	0	1
2	0	0	1	1	1	0
3	1	1	1	0	0	0
4	1	1	0	0	1	1
5	0	1	0	1	1	0
6	0	0	1	1	0	0
7	1	0	0	1	0	0
8	1	1	0	0	0	0
9	1	1	0	0	0	0
10	0	0	1	1	1	1
11	0	1	1	1	1	0
12	0	1	0	1	0	0
13	1	0	0	0	0	0
14	0	1	0	0	0	0
15	0	1	0	1	1	0
16	0	1	1	1	0	0
17	0	1	0	1	0	0
18	1	1	0	0	1	0
19	0	1	1	1	1	0
20	1	1	1	0	1	0
21	0	1	1	1	1	0
22	0	1	1	1	0	0
23	0	0	0	0	1	0
24	1	0	0	0	0	0
25	1	1	1	1	0	0

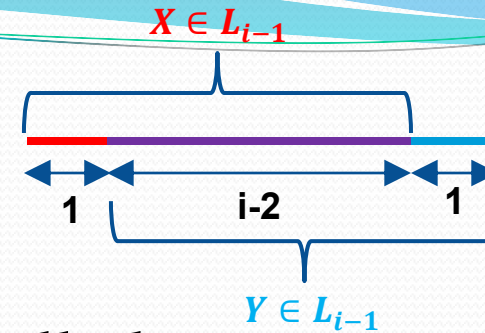
{a, b}	{a, c}	{a, d}	{a, e}	{b, c}
1	0	0	0	0
{b, d}	{b, e}	{c, d}	{c, e}	{d, e}
0	0	0	0	0

Tid	a	b	c	d	e	f
1	1	1	0	0	0	1
2	0	0	1	1	1	0
3	1	1	1	0	0	0
4	1	1	0	0	1	1
5	0	1	0	1	1	0
6	0	0	1	1	0	0
7	1	0	0	1	0	0
8	1	1	0	0	0	0
9	1	1	0	0	0	0
10	0	0	1	1	1	1
11	0	1	1	1	1	0
12	0	1	0	1	0	0
13	1	0	0	0	0	0
14	0	1	0	0	0	0
15	0	1	0	1	1	0
16	0	1	1	1	0	0
17	0	1	0	1	0	0
18	1	1	0	0	1	0
19	0	1	1	1	1	0
20	1	1	1	0	1	0
21	0	1	1	1	1	0
22	0	1	1	1	0	0
23	0	0	0	0	1	0
24	1	0	0	0	0	0
25	1	1	1	1	0	0

✓				✓
{a, b}	{a, c}	{a, d}	{a, e}	{b, c}
8	3	2	3	8
{b, d}	{b, e}	{c, d}	{c, e}	{d, e}
10	8	9	6	7
✓	✓	✓	✓	✓



From L_{i-1} to C_i

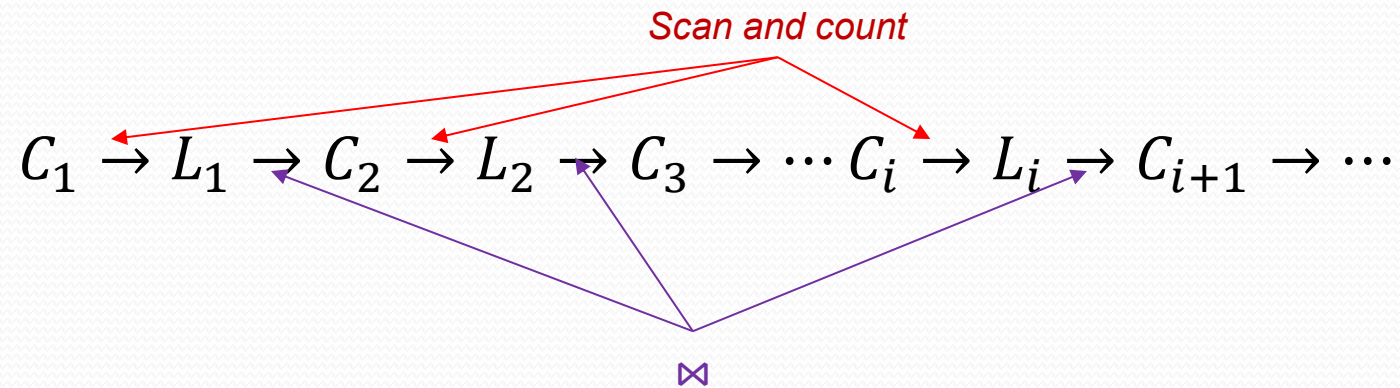


- The Apriori Algorithm uses a function called *apriori-gen* to generate C_i given an L_{i-1} .
- *apriori-gen* considers all pairs of $(i-1)$ -itemsets X and Y from L_{i-1} that share $i-2$ common items.
- E.g., to generate C_3 from L_2 , we need X, Y pairs in L_2 that share 1 common item. $\{b, d\}, \{c, d\}$ is an example pair.
- For each such X, Y pair, *apriori-gen* generates the i -itemset $Z = X \cup Y$.

From L_{i-1} to C_i

- Each such itemset Z generated will be checked to see if all of its size- $(i-1)$ subsets are in L_{i-1} . If yes, Z is put into C_i .
- We write: $C_i = L_{i-1} \bowtie L_{i-1}$ to represent the apriori-gen procedure
- Example:
 - From $\{b, d\}$ and $\{c, d\}$, *apriori-gen* generates the 3-itemset, $\{b, c, d\}$.
 - Since all $\{b, c\}$, $\{c, d\}$, and $\{b, d\}$ are in L_2 , $\{b, c, d\}$ is included in C_3 .
 - From $\{a, b\}$ and $\{b, c\}$, *apriori-gen* generates the 3-itemset, $\{a, b, c\}$.
 - Since $\{a, c\}$ is not in L_2 , $\{a, b, c\}$ is not included in C_3 .

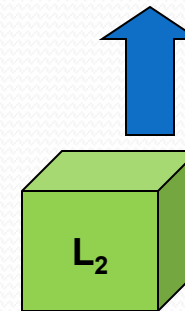
Apriori Algorithm



Tid	a	b	c	d	e	f
1	1	1	0	0	0	1
2	0	0	1	1	1	0
3	1	1	1	0	0	0
4	1	1	0	0	1	1
5	0	1	0	1	1	0
6	0	0	1	1	0	0
7	1	0	0	1	0	0
8	1	1	0	0	0	0
9	1	1	0	0	0	0
10	0	0	1	1	1	1
11	0	1	1	1	1	0
12	0	1	0	1	0	0
13	1	0	0	0	0	0
14	0	1	0	0	0	0
15	0	1	0	1	1	0
16	0	1	1	1	0	0
17	0	1	0	1	0	0
18	1	1	0	0	1	0
19	0	1	1	1	1	0
20	1	1	1	0	1	0
21	0	1	1	1	1	0
22	0	1	1	1	0	0
23	0	0	0	0	1	0
24	1	0	0	0	0	0
25	1	1	1	1	0	0

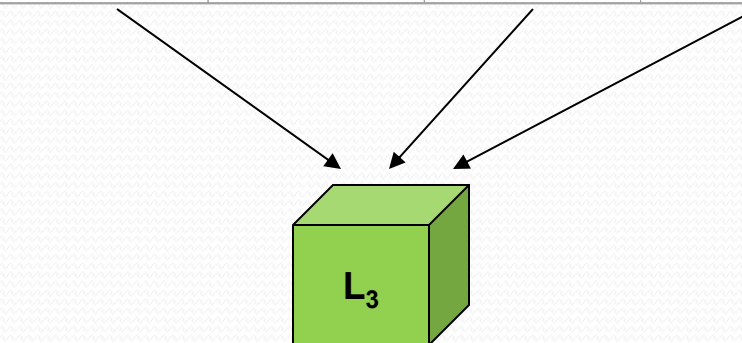
 C_3

{b, c, d}	{b, c, e}	{b, d, e}	{c, d, e}
0	0	0	0



Tid	a	b	c	d	e	f
1	1	1	0	0	0	1
2	0	0	1	1	1	0
3	1	1	1	0	0	0
4	1	1	0	0	1	1
5	0	1	0	1	1	0
6	0	0	1	1	0	0
7	1	0	0	1	0	0
8	1	1	0	0	0	0
9	1	1	0	0	0	0
10	0	0	1	1	1	1
11	0	1	1	1	1	0
12	0	1	0	1	0	0
13	1	0	0	0	0	0
14	0	1	0	0	0	0
15	0	1	0	1	1	0
16	0	1	1	1	0	0
17	0	1	0	1	0	0
18	1	1	0	0	1	0
19	0	1	1	1	1	0
20	1	1	1	0	1	0
21	0	1	1	1	1	0
22	0	1	1	1	0	0
23	0	0	0	0	1	0
24	1	0	0	0	0	0
25	1	1	1	1	0	0

☒	☒	☒	
{b, c, d}	{b, c, e}	{b, d, e}	{c, d, e}
6	4	5	5



Example

- $L_3 = \{ \{b, c, d\}, \{b, d, e\}, \{c, d, e\} \}$
- The only 4-itemset that can be generated is $\{b, c, d, e\}$
- Note that $\{b, c, e\} \notin L_3$
- Since not all size-3 subsets of $\{b, c, d, e\}$ are in L_3 , C_4 is empty.
- Since C_4 is empty, L_4 must be empty. Apriori thus terminates.
- Notes:
 - D has 6 items, so there are $2^6 - 1 = 63$ possible non-empty itemsets
 - Apriori has counted the supports of 20 itemsets
 - In practice, the reduction of itemset counting is much more drastic than the example shows.

The Algorithms

Algorithm Apriori

Begin

Scan the dataset D once, and put all frequent 1-itemsets in L_1 ;

$k = 1$;

while ($L_k \neq \text{NULL}$) {

$C_{k+1} = \text{Apriori-gen}(L_k)$;

 For each transaction T in D do

 For each c in C_{k+1} that is a subset of T do

 Increment c 's support count;

$L_{k+1} = \text{itemsets in } C_{k+1} \text{ whose support count} \geq \rho_s * N$;

$k++$;

}

End

The Algorithms

Function Apriori-gen (L_k)

Begin

$C_{k+1} = \text{NULL};$

For each itemset X in L_k

For each itemset Y in L_k

If ($X \cap Y$ contains $k-1$ items) {

Let $Z = X \cup Y;$

If every size k subset of Z is in L_k

$C_{k+1} = C_{k+1} \cup \{Z\};$

}

Return $C_{k+1};$

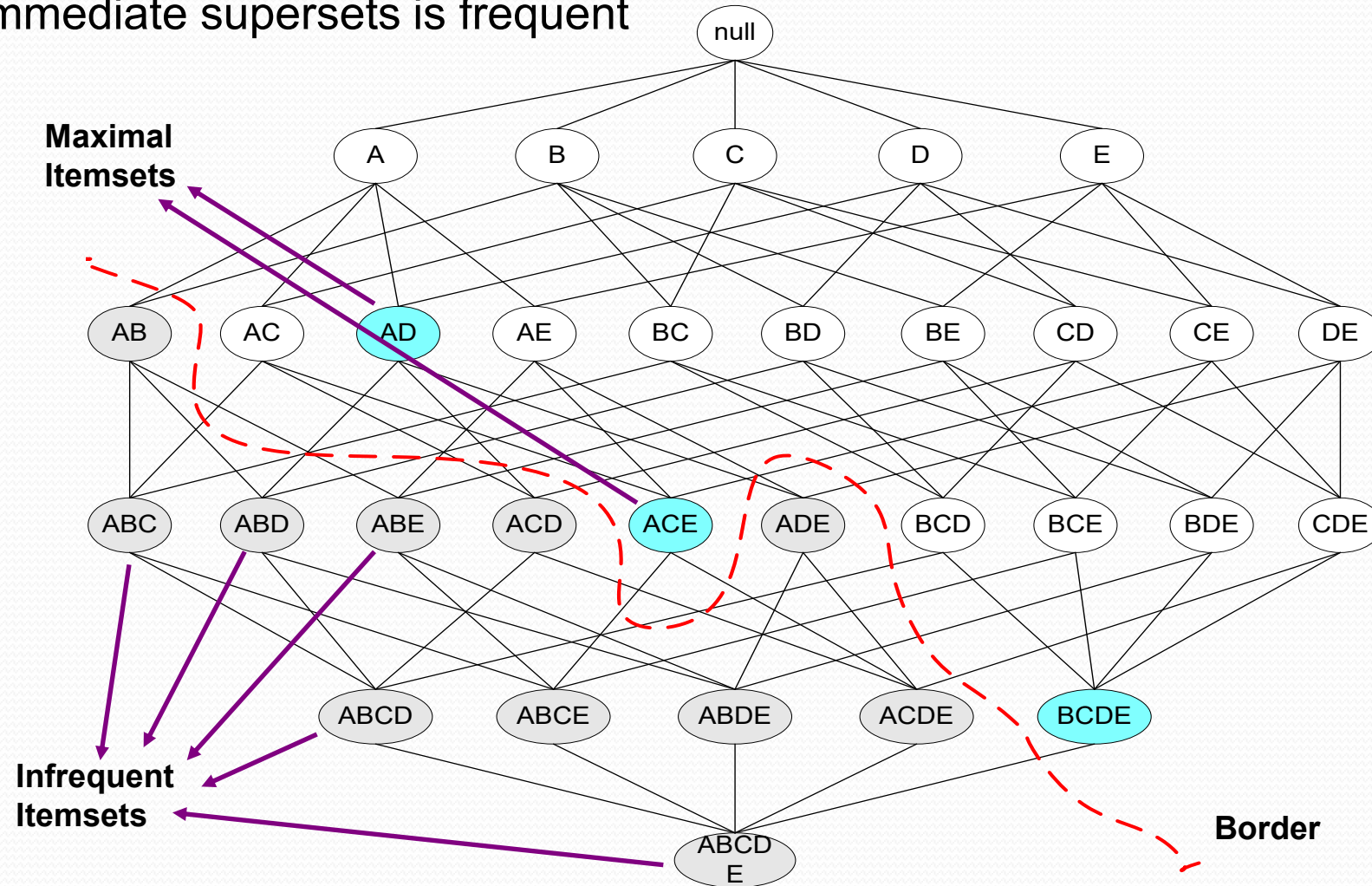
End

Compact representation

- The first step of association rule mining is to find all frequent itemsets and their support counts. (The set L .)
- L is typically huge!
- We need a more compact representation.
 - Maximal frequent itemsets
 - Closed frequent itemsets

Maximal Frequent Itemset

An itemset is *maximal frequent* if (1) it is frequent and (2) none of its immediate supersets is frequent



Maximal Frequent Itemset

- Theorem:
An itemset X is frequent iff X is a subset of some maximal frequent itemset. i.e.,

$$L = \{X \mid X \subseteq Z, \text{ where } Z \text{ is maximal frequent}\}$$

- Implication: Storing Z (a smaller set) is enough to derive L (a larger set).

Closed Itemset

- An itemset X is *closed* if none of its immediate supersets has the same support as X

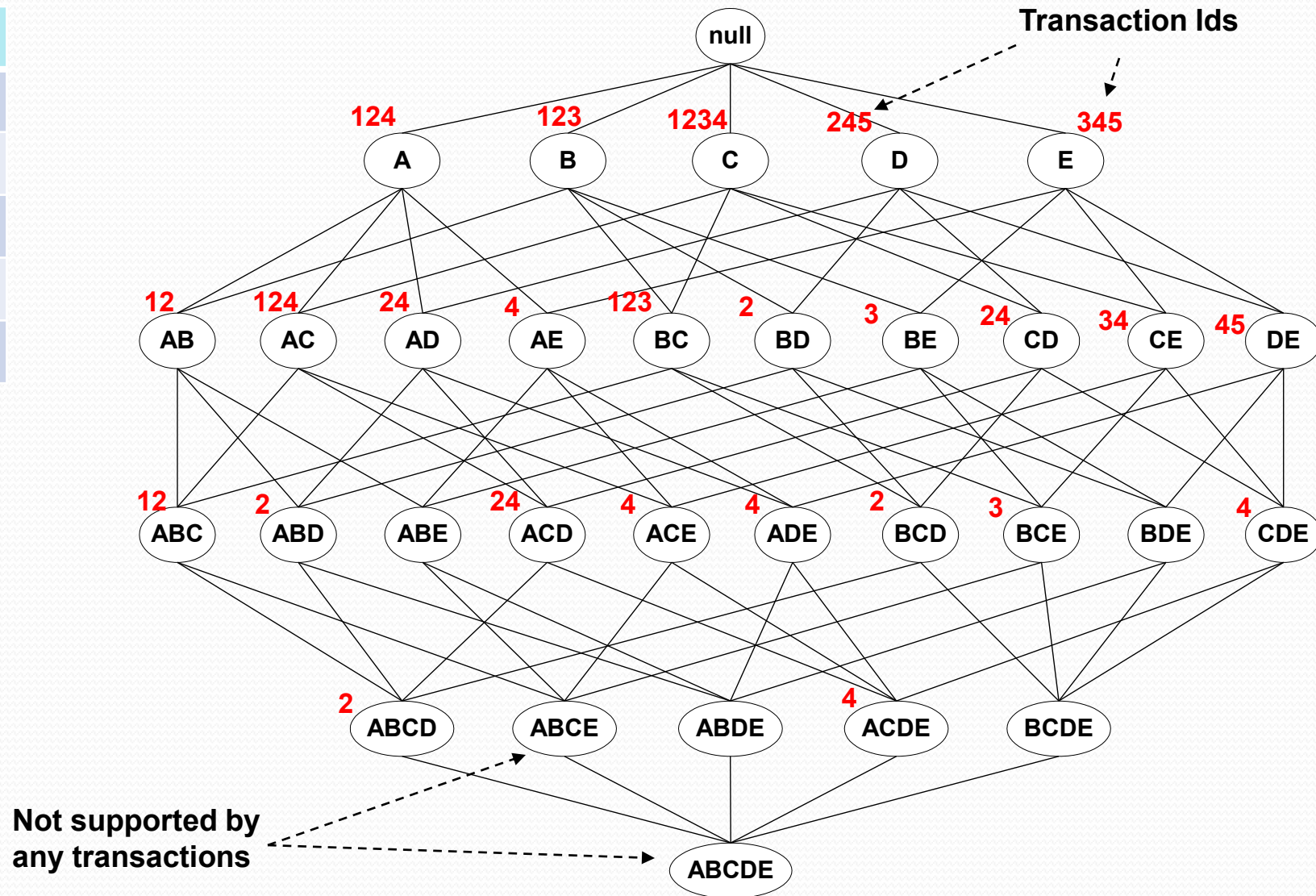
itemset	support	itemset	support
{a}	11	{b, d}	10
{b}	18	{c, d}	9
{c}	11	{c, e}	6
{d}	14	{d, e}	7
{e}	11	{c, d, e}	5
{a, b}	8	{b, c}	8
{b, e}	8	{b, d, e}	5
{b, c, d}	6		

TID	Items
1	{A,B}
2	{B,C,D}
3	{A,B,C,D}
4	{A,B,D}
5	{A,B,C,D}

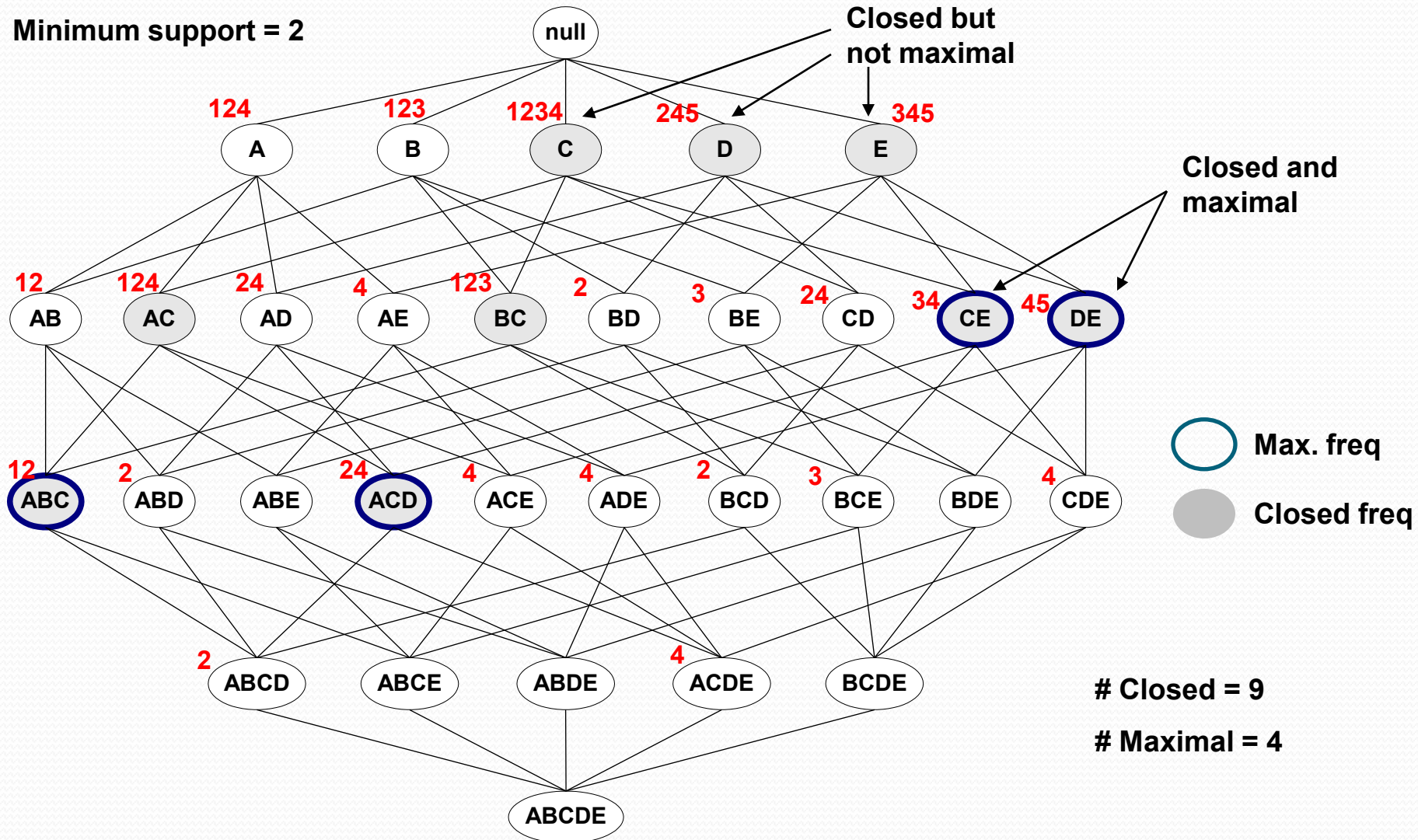
Itemset	Support
{A}	4
{B}	5
{C}	3
{D}	4
{A,B}	4
{A,C}	2
{A,D}	3
{B,C}	3
{B,D}	4
{C,D}	3

Itemset	Support
{A,B,C}	2
{A,B,D}	3
{A,C,D}	2
{B,C,D}	3
{A,B,C,D}	2

TID	Items
1	ABC
2	ABCD
3	BCE
4	ACDE
5	DE



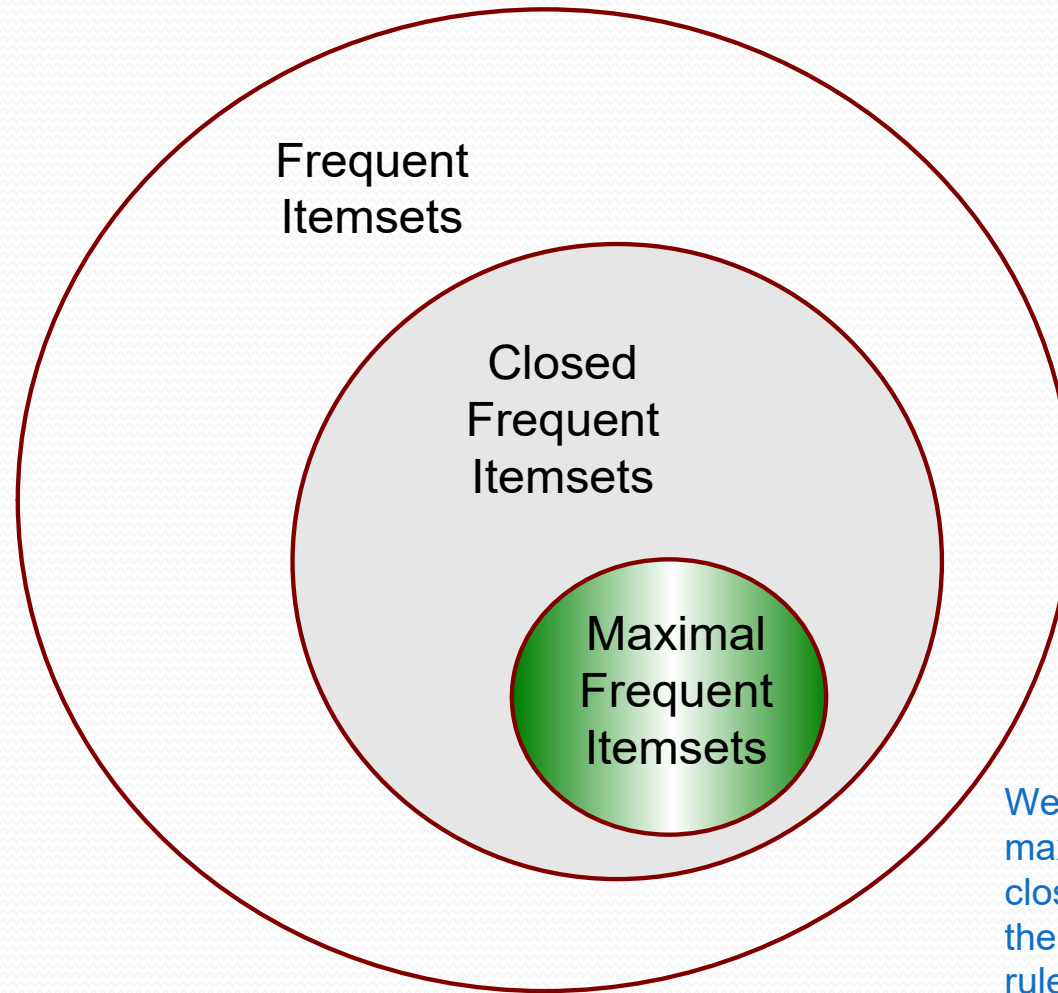
TID	Items
1	ABC
2	ABCD
3	BCE
4	ACDE
5	DE



Maximal/Closed Frequent Itemsets

- To deduce all the frequent itemsets and their support counts, we only need:
 1. The set of all maximal frequent itemsets.
 2. The support counts of all closed frequent itemsets.

Maximal vs Closed Itemsets



We only need to know the max. freq. itemsets and the closed frequent itemsets and their support counts to find all rules

Maximal frequent itemsets can infer all frequent itemsets

Closed frequent itemsets can infer all frequent itemsets and their supports

High-utility itemset

- Instead of support, we can evaluate the importance of an itemset by its *utility*.
- Data model:
 - Each transaction T records the items and their quantities sold.
Let $q(a, T)$ = quantity of a sold in transaction T
 - Each item a is associated with a unit-profit $p(a)$.

High-utility itemset

- Problem definition:

- Given a transaction T and an itemset X , if $X \subseteq T$, then the profit of X in T :

$$\sigma(X, T) = \sum_{\substack{a \in X \\ X \subseteq T}} (p(a) \times q(a, T))$$

- The utility of itemset X in a dataset D :

$$\pi(X, D) = \sum_{T \in D} \sigma(X, T)$$

- Objective is to find all itemsets X s.t. $\pi(X, D) \geq \text{min}_u$, where min_u is a minimum utility threshold.

D

Tid	a	b	c	d	e	f
1	5	0	0	0	3	1
2	1	2	3	1	0	9
3	2	0	1	1	2	2
4	0	4	2	8	5	1
5	2	1	0	0	0	1

Utility of {a,e}

$$\begin{aligned}
 \pi(\{a, e\}, D) &= (5 \times 3 + 3 \times 5) + (2 \times 3 + 2 \times 5) \\
 &= 46
 \end{aligned}$$

Item	Profit (p)
a	3
b	1
c	2
d	8
e	5
f	2